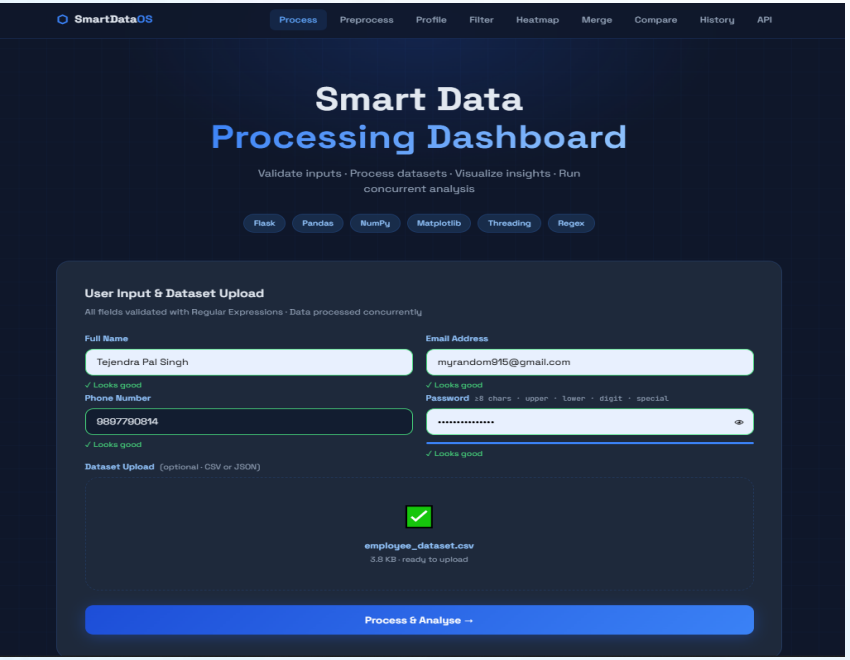
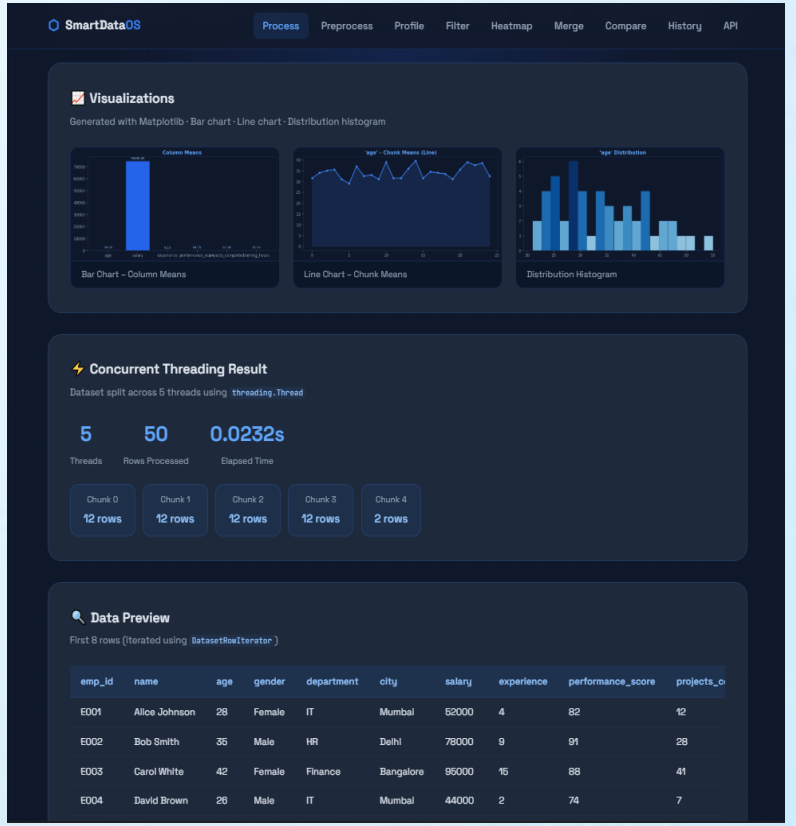
DEMO

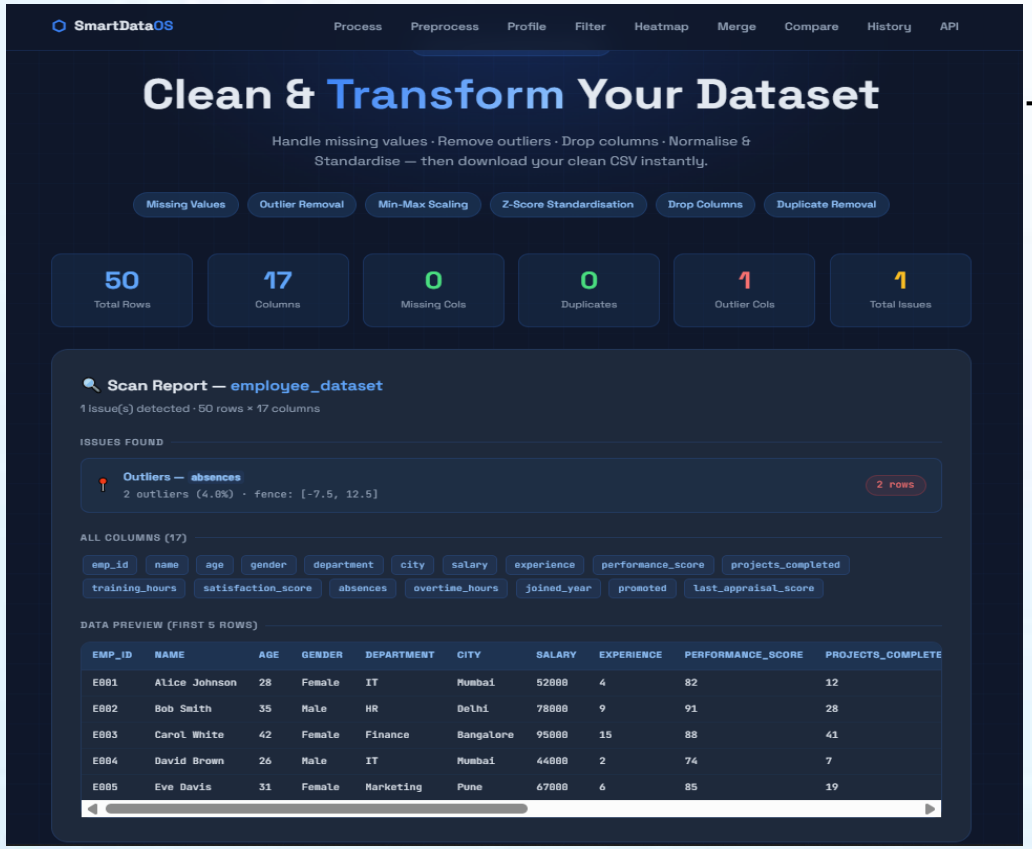


→ Upload Your Dataset

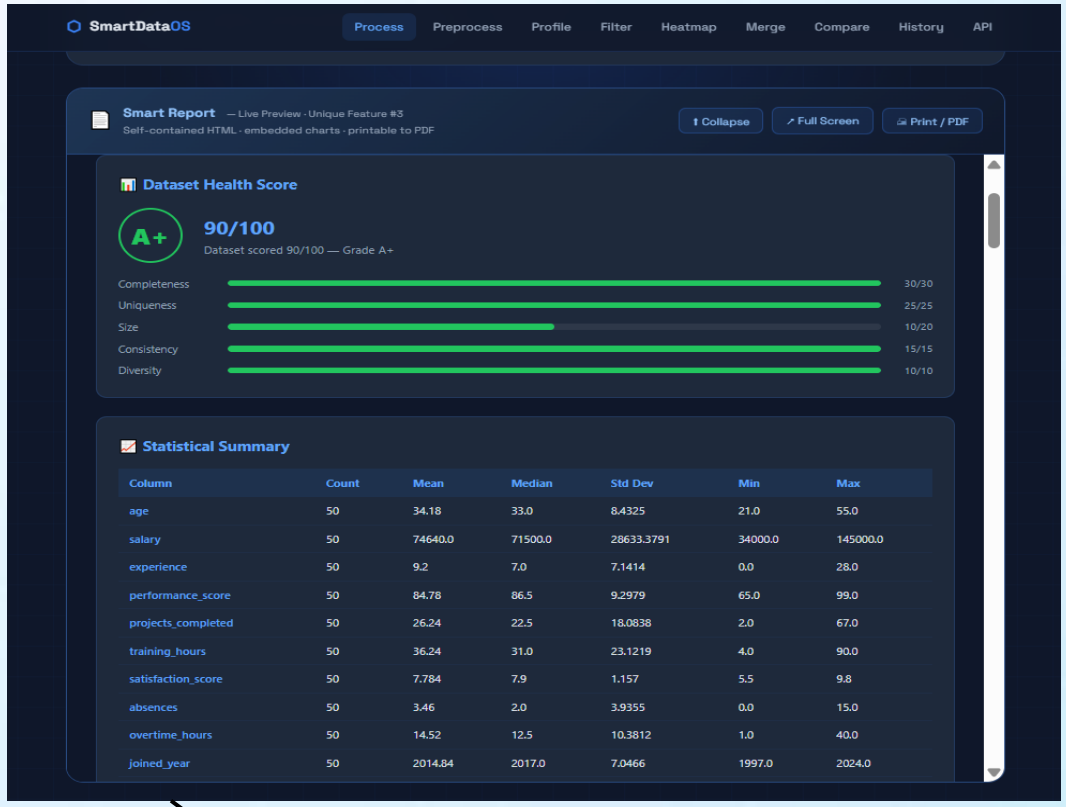
Get instant Details Along with a Printable Report of Dataset showing all columns count, mean , deviations etc..



DEMO



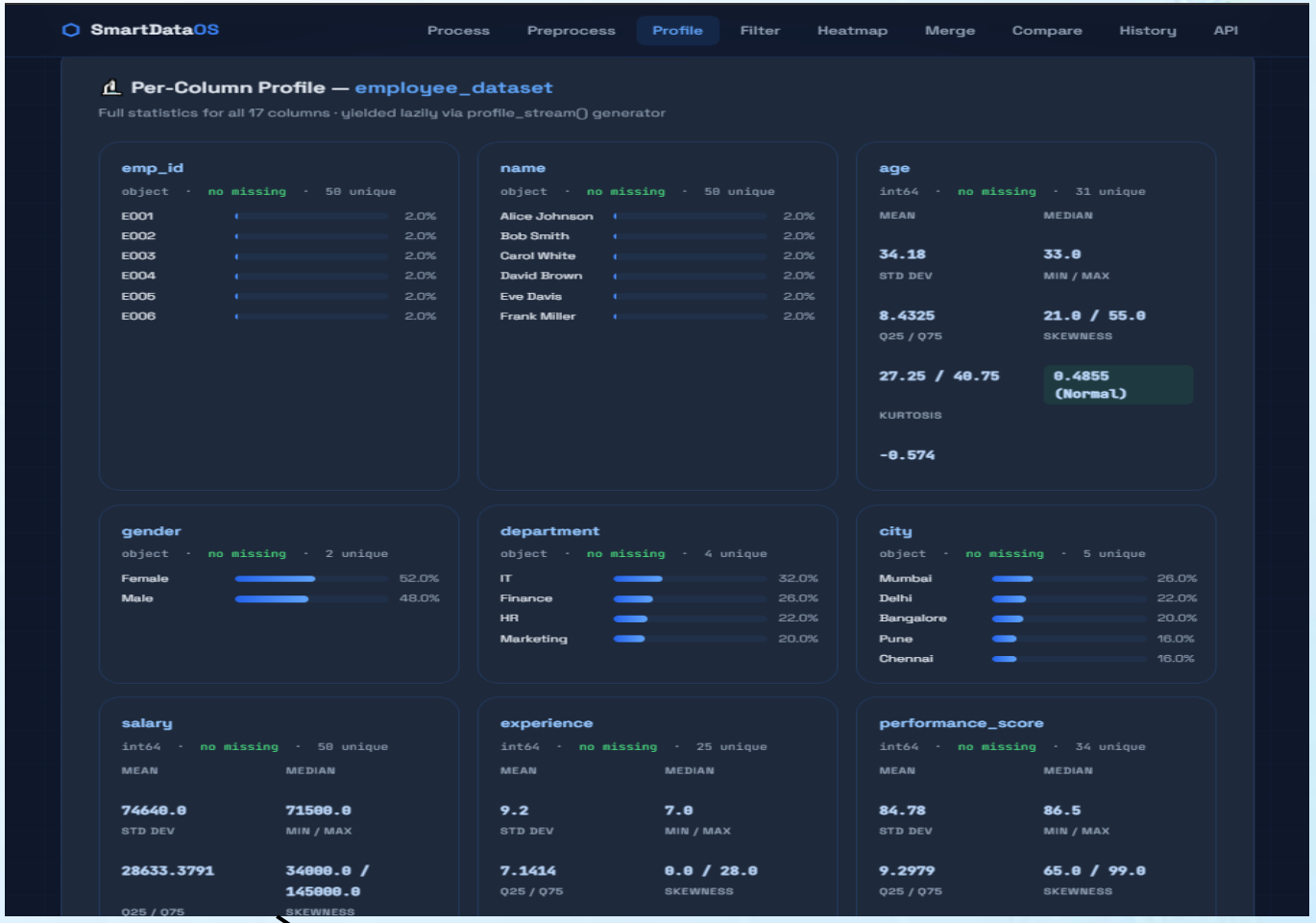
→ Clean and Transform your data set by removing outliers , missing values and errors by just uploading it



→ Get printable report and HealthScore of dataset

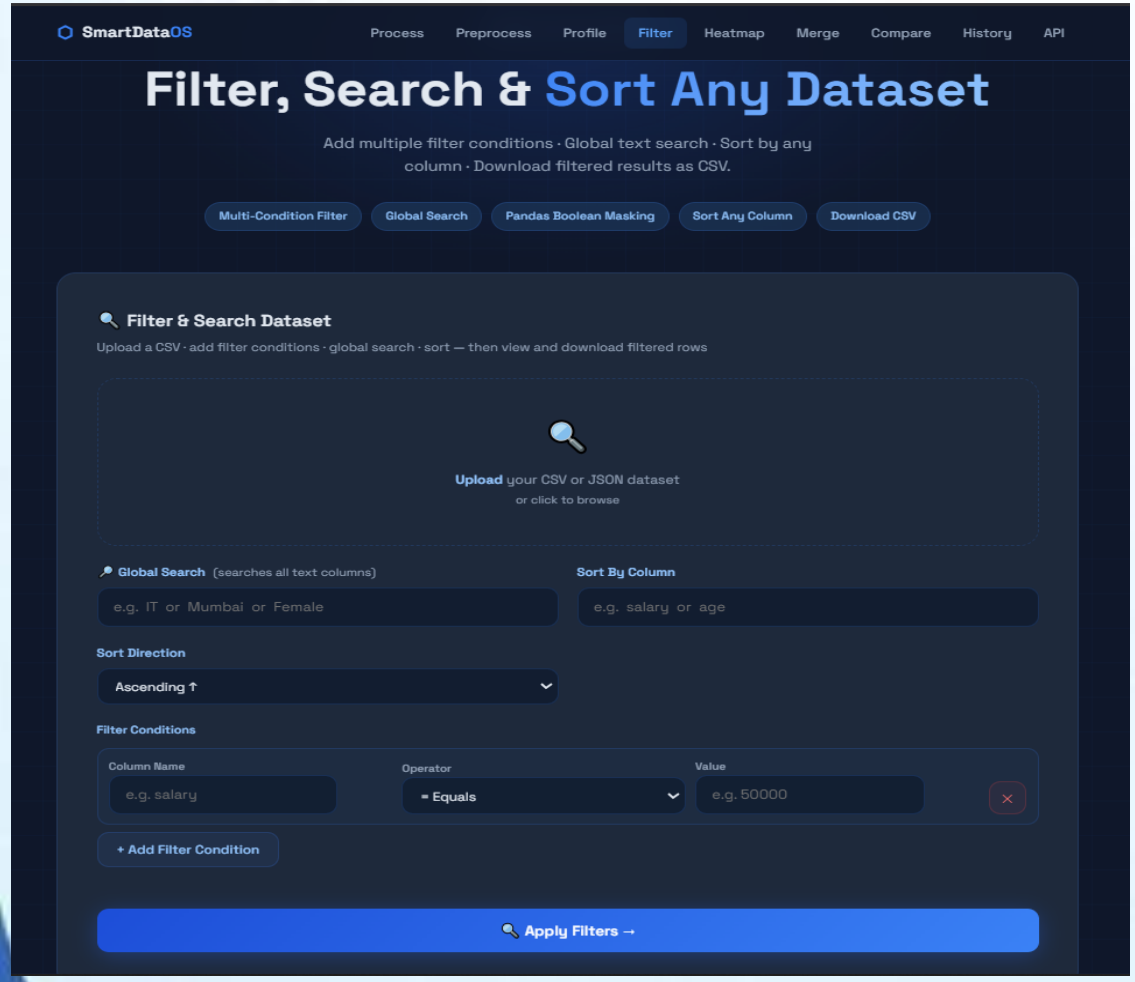
DEMO

→ Get Details of each and every Column

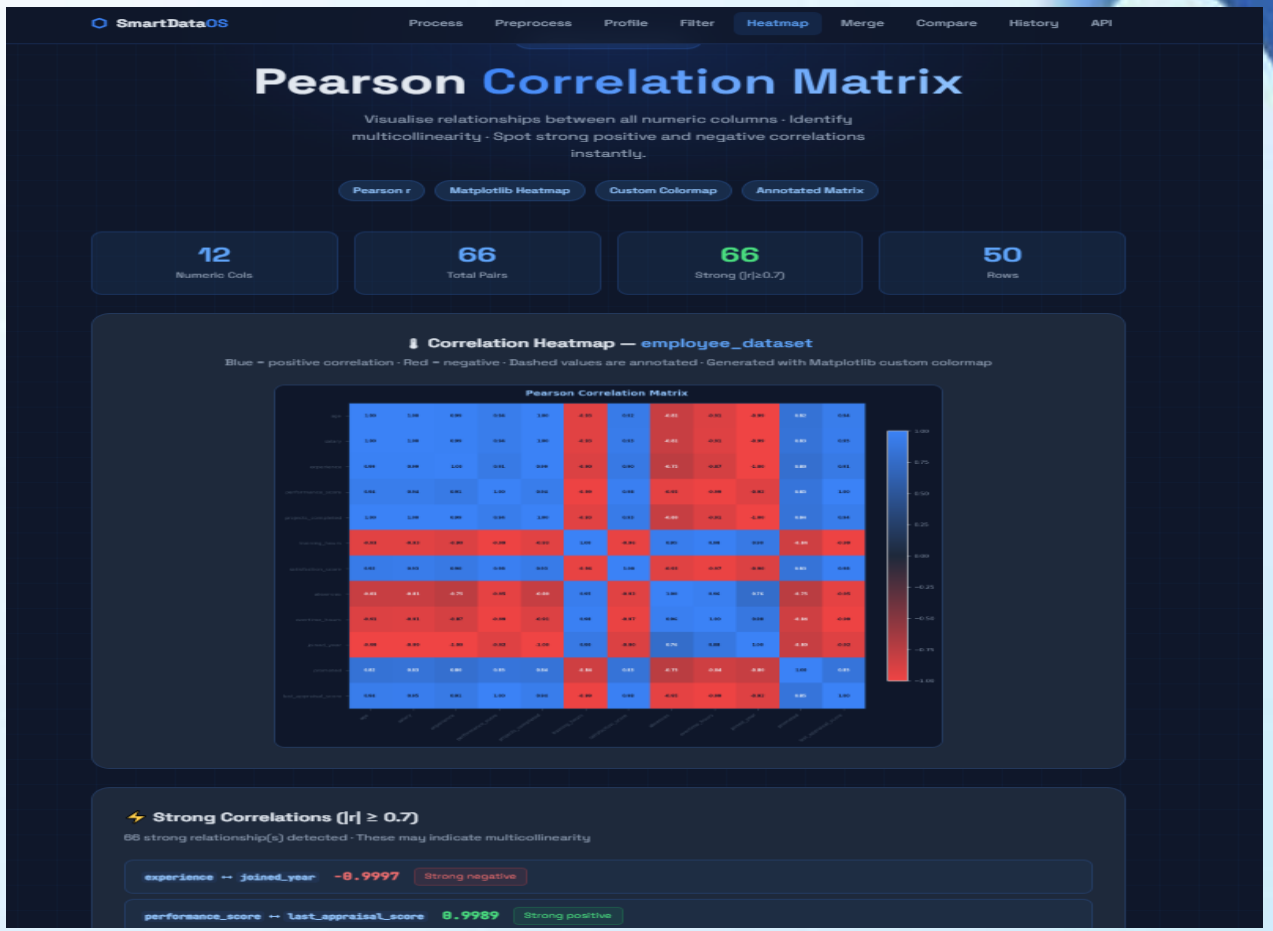


↘ Each Column Detail Separately

DEMO



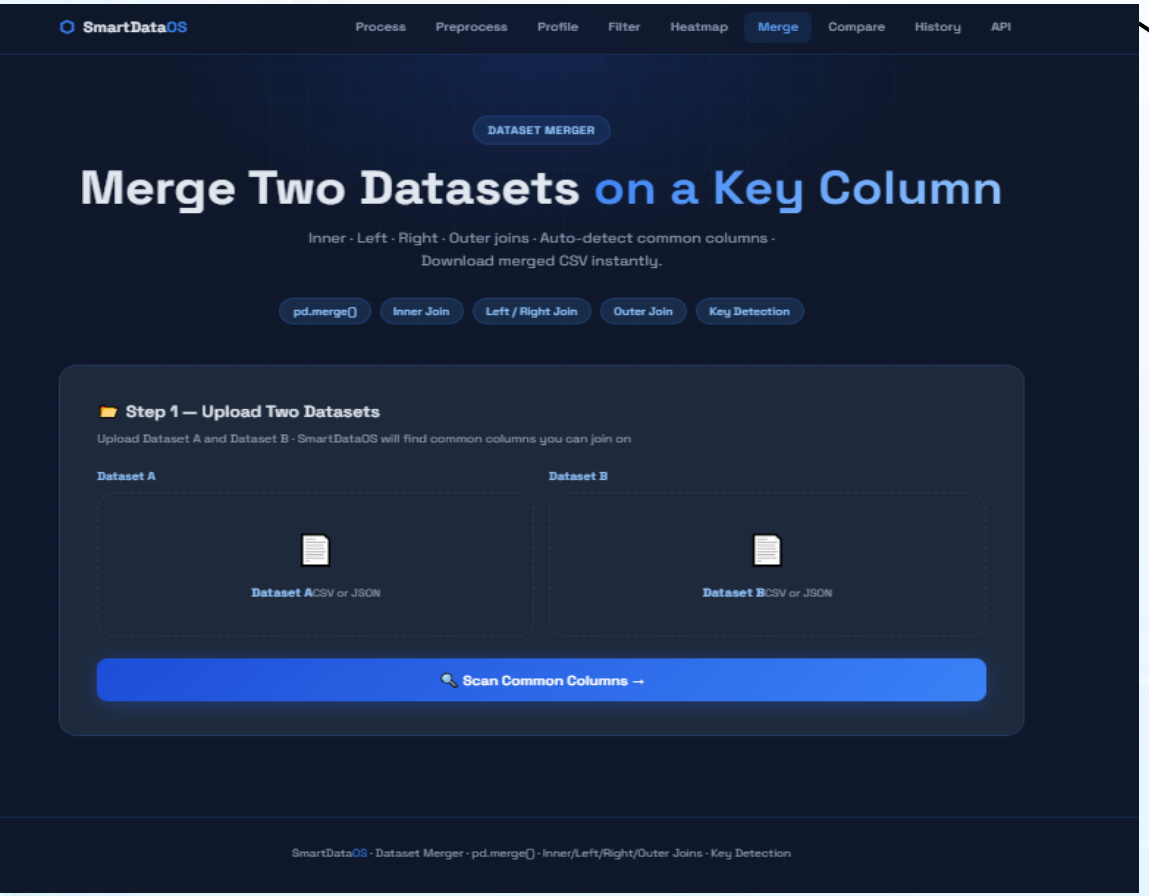
Filter and Sort any Dataset Based on Your Conditions



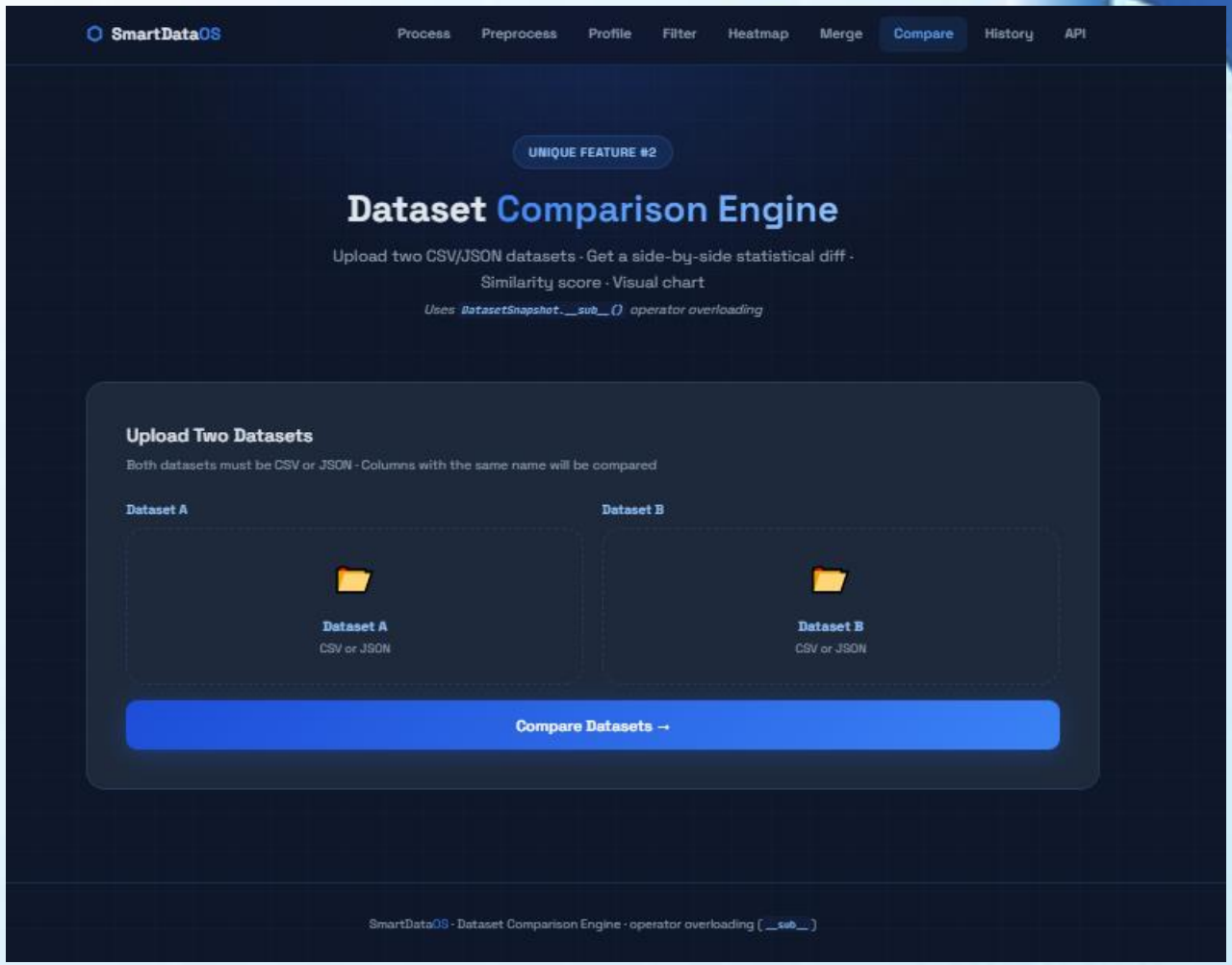
Get a Correlation matrix to find how different Columns are co-related

DEMO

Merge any Two Dataset



Compare any Two Dataset



COMPLETE PROJECT DOCUMENTATION

SmartDataOS

Smart Data Processing & Validation Dashboard

Framework	Python 3 · Flask · Pandas · NumPy · Matplotlib
Total Pages	9 pages covering all features, modules, routes, and concepts
Features	9 major features · 19+ Python concepts · 19 Flask routes
Modules	14 Python modules across modules/ and utils/ folders
Sample Data	employee_dataset.csv (50 rows x 17 cols) · it_dept.csv · finance_dept.csv

TABLE OF CONTENTS

■ 1 Project Overview

What Smart

SmartDataOS is a full-stack web application built with Python and Flask. It lets you upload any CSV or JSON dataset, analyse it, clean it, visualise correlations, filter rows, merge multiple files, compare datasets, and download results — all from a single browser-based dashboard. No coding required for the end user.

How It Works — Simple Flow

1. Open your browser and go to http://127.0.0.1:5000
2. Fill in your name, email, phone and password — all validated with Regex
3. Upload a CSV or JSON file using drag-and-drop
4. Click "Process & Analyse" — SmartDataOS does everything automatically
5. View statistics, charts, AI insights, health score and a live HTML report
6. Use the navbar to access Preprocess, Profile, Filter, Heatmap, Merge pages

Technology Stack

Flask	Web framework — handles all HTTP routes, form data and file uploads
Pandas	Loads CSV/JSON into DataFrames, computes stats, filters and merges data
NumPy	Mean, median, std, percentiles, skewness, kurtosis, correlation matrix
Matplotlib	Generates all charts server-side using the Agg (non-display) backend
Threading	Splits dataset processing across multiple threads concurrently
json	Reads and writes all processing records to data/datasets.json
re	Pre-compiled regex patterns validate all four user input fields
base64	Encodes chart PNGs into the self-contained HTML report
os/sys/datetime	Standard library modules used throughout for paths, versions, timestamps

■ 2 Getting Started

Installation,

Requirements

Python	3.10 or higher (tested on Python 3.14 on Windows)
pip	comes with Python — used to install packages
Browser	Any modern browser: Chrome, Firefox, Edge, Safari
Disk space	About 50 MB including Python packages

Step-by-Step Setup

```
venv\Scripts\activate          # Windows                                    source v
```

requirements.txt Contents

```
pandas==2.1.4                                                              matplotl
```

Important: Always run "python app.py" from INSIDE the smart_data_system/ folder. Running from a different directory may cause file path errors with charts and reports.

■ 3 Feature 1 — Process & Analyse

Home page

This is the main feature of SmartDataOS. You fill in a form, upload a dataset, and the system automatically runs validation, statistics, charts, threading, AI insights, health scoring, and generates an HTML report — all in one click.

Step 1 — User Input Form (Home Page)

The form has 4 fields, each validated in real-time as you type using pre-compiled regular expressions:

Full Name	^[A-Za-z\s\-']{2,60}$ — letters, spaces, hyphens, apostrophes (2-60 chars)
Email	^[\w.+\-]+@[\w\-]+\.[a-zA-Z]{2,}$ — standard email format
Phone	^\+?[\d\s\-\(\)]{7,15}$ — optional +, digits, spaces, hyphens
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]) — min 8 chars, mixed
Dataset	Optional CSV or JSON file · drag and drop supported · max 10 MB

Step 2 — What Happens After You Click Submit

The /process route runs these 7 steps in order:

→	**1. Load Dataset**	pandas pd.read
→	**2. Statistics**	NumPy comput
→	**3. Charts**	Matplotlib gene
→	**4. Threading**	Dataset split ac
→	**5. AI Insights**	InsightsEngine
→	**6. Health Score**	Dataset graded
→	**7. HTML Report**	generate_repor

Dashboard Results

Stat Cards	50 rows · 17 cols · 12 numeric · 0 missing · 5 batches — shown at a glance
Column Stats	Full table: count, mean, median, std, min, max, Q25, Q75 for every numeric column
3 Charts	Bar chart of means · Line chart of chunk means · Distribution histogram
Threading Panel	5 threads · 50 rows · elapsed time · chunk breakdown (12/12/12/12/2 rows)
Data Preview	First 8 rows shown in a scrollable table using DatasetRowIterator
AI Insights	80+ plain-English findings with severity colour-coding (critical/warning/good/insight)
Health Score	Grade circle (A+) with 5 animated progress bars and score out of 100
Live Report	Embedded iframe showing the full HTML report with collapse/print/fullscreen buttons

■ 4 Feature 2 — Data Preprocessing

Clean, trans

The Preprocess page scans your dataset for data quality issues and lets you fix them with one click. After cleaning, you can download the processed CSV file.

How to Use It

● **Upload**	Go to Preproce
● **Scan**	SmartDataOS s
● **Configure**	Choose which
● **Transform**	Re-upload the
● **Download**	Click the green

Available Operations

Fill Missing — Mean	Replaces null values with the column arithmetic mean (numeric cols only)
Fill Missing — Median	Replaces null values with the column median (better for skewed data)
Fill Missing — Mode	Replaces null values with the most frequent value (works on text too)
Fill Missing — Zero	Replaces numeric nulls with 0
Drop Null Rows	Removes any row that contains at least one missing value
Remove Outliers (IQR)	Removes rows where any value is outside Q25-1.5*IQR or Q75+1.5*IQR
Drop Columns	Type column names separated by commas — those columns are removed
Min-Max Scaling	Rescales numeric values to range 0.0 to 1.0 — formula: (x-min)/(max-min)
Z-Score Standardisation	Shifts data to mean=0, std=1 — formula: (x-mean)/std
Remove Duplicates	Keeps first occurrence, removes all exact duplicate rows

■ 5 Feature 3 — Data Profiler

Deep per-co

The Data Profiler gives you a complete breakdown of every single column in your dataset. It goes beyond basic statistics — it computes skewness and kurtosis for numeric columns, and shows top value counts with percentage bars for text columns. A mini histogram or bar chart is auto-generated for each column.

What You Get Per Numeric Column

Count	Number of non-null values in the column
Missing	Number and percentage of null/NaN values
Unique	Number of distinct values in the column
Mean	Arithmetic average — sum of values divided by count
Median	Middle value when sorted — less affected by outliers than mean
Std Dev	Standard deviation — how spread out values are from the mean
Min / Max	Smallest and largest values in the column
Q25 / Q75	25th and 75th percentile (quartiles) — used in IQR outlier detection
Skewness	Measures asymmetry: positive = right tail, negative = left tail, 0 = symmetric
Kurtosis	Measures tail heaviness: high kurtosis = more outliers than a normal distribution

What You Get Per Text Column

Top 8 Values	Most frequent values shown with count and percentage bar
Mode	The single most common value in the column
Unique Count	How many distinct values exist (cardinality)
Missing Count	How many nulls are in this column

How Skewness Is Labelled

Right-skewed	Skew > 0.5 — tail points right, most values are small (e.g. salary, absences)
Left-skewed	Skew < -0.5 — tail points left, most values are large
Normal	-0.5 to 0.5 — approximately symmetrical distribution

■ 6 Feature 4 — Smart Filter & Search

Multi-condit

The Filter page lets you search and filter any dataset without writing any code. Add multiple filter conditions, search across all text columns at once, sort by any column, and download only the rows that match.

Filter Operators Available

= Equals	Row must have exactly this value in the column
Not Equals	Row must NOT have this value in the column
Contains	Column value must contain this text (case-insensitive, text columns)
Greater Than	Column value must be strictly greater than the number entered
Greater or Equal	Column value must be greater than or equal to the number
Less Than	Column value must be strictly less than the number entered
Less or Equal	Column value must be less than or equal to the number
Between	Enter two numbers separated by comma (e.g. 50000,100000)
Is Empty	Only return rows where this column has a null/missing value
Is Not Empty	Only return rows where this column has an actual value

Example: Filter employee_dataset.csv

```
Filter 1:  column=experience   operator=between    value=5,15
```

You can add as many filter conditions as you want. All conditions work together — a row must satisfy ALL conditions to appear in the results. The first 50 matching rows are shown on screen; the full filtered dataset is available to download as a CSV.

■ 7 Feature 5 — Correlation Heatmap

Pearson co

The Heatmap page computes the Pearson correlation coefficient between every pair of numeric columns and visualises it as a colour-coded matrix. Blue means a positive relationship, red means negative, and the number shows the exact strength.

Understanding Correlation Values

r = +1.0	Perfect positive — as one column goes up, the other always goes up too
r = +0.7 to 1	Strong positive correlation — highlighted in green
r = +0.4 to 0.7	Moderate positive — highlighted in yellow/amber
r = 0 to 0.4	Weak or no relationship
r = -0.4 to -0.7	Moderate negative — one goes up, other tends to go down
r = -0.7 to -1	Strong negative correlation — highlighted in red
r = -1.0	Perfect negative — as one goes up, the other always goes down

Real Example — employee_dataset.csv

experience ↔ joined_year	r = -1.000 — Perfect negative (more experience = older join year)
salary ↔ projects_completed	r = +0.997 — Strong positive (high salary = more projects)
performance_score ↔ last_appraisal	r = +0.999 — Near perfect positive
satisfaction_score ↔ absences	r = -0.929 — Strong negative (unhappy = more absences)

Why Correlation Matters

High correlations (|r| >= 0.7) can indicate multicollinearity — two columns that contain essentially the same information. In machine learning, keeping highly correlated features can reduce model accuracy. The heatmap makes it instantly visible which columns to consider removing.

■ 8 Feature 6 — Dataset Merger

Join two dat

The Merge page lets you combine two separate CSV or JSON files into one. It works like a database JOIN — you pick a column that exists in both files and SmartDataOS combines the rows where the values match.

The 4 Join Types Explained Simply

Inner Join	ONLY keeps rows where the key value exists in BOTH files. Safest, smallest result.
Left Join	Keeps ALL rows from Dataset A. Adds matching data from B where it exists, empty where it does not.
Right Join	Keeps ALL rows from Dataset B. Adds matching data from A where it exists, empty where it does not.
Outer Join	Keeps ALL rows from BOTH files. Missing values filled with NaN where no match exists.

Example — Merge employee_dataset with finance_dept

```
# Only the 13 employees that appear in BOTH files are kept                    # Result
```

If the same column name exists in both files (other than the key), SmartDataOS automatically renames them — adding _A suffix for Dataset A and _B suffix for Dataset B to avoid confusion.

■ 9 Features 7 & 8 — Compare & History

Side-by-side

Feature 7 — Dataset Comparison

The Compare page uses Python operator overloading — when you upload two files and click Compare, the code runs snap_a - snap_b, which triggers DatasetSnapshot.__sub__() and automatically calls ComparisonEngine.compare().

Similarity Score	0-100 score based on mean percentage difference across all shared numeric columns
Column Availability	Shows which columns exist only in A, only in B, or in both
Per-Column Diff	For each shared column: mean A, mean B, % change, and a plain-English verdict
Visual Chart	Grouped bar chart comparing column means of both datasets side by side
Example result	employee_dataset vs finance_dept: Score 36/100 · salary +15.8% · experience +31.3%

Feature 8 — Processing History

Every time you process a dataset, the result is saved automatically to data/datasets.json using Python's json module. The History page reads this file and shows all past records in a card grid.

What is saved	User name, email, phone, timestamp, dataset shape, column stats, chart paths
Record ID	Auto-generated unique ID like rec_0001, rec_0002, etc.
Thumbnails	Small preview images of the bar and line charts for each past record
Delete	Click the X on any card to delete that record (POST /history/delete/id)
Raw JSON API	Visit /api/stats to see all records as raw JSON in the browser
File	data/datasets.json — human-readable JSON array of all records

■ 10 Python Concepts Reference

All 19+ conc

#	Concept	File	How It Is Used
01	**Custom Iterator**	utils/iterators.py	__iter__ + __next__ — DatasetRowIterator walks rows in batches
02	**Generator (×3)**	utils/generators.py	chunk_dataset · stats_stream · json_record_generator — all use yield
03	**Generator (profile_stream)**	modules/profiler.py	Yields one column profile dict at a time — lazy evaluation
04	**Generator (filtered_row_gen)**	modules/filter_engine.py	Yields filtered rows one at a time after filtering is applied
05	**Decorator @log_call**	utils/decorators.py	Logs every function call with args and return value to app.log
06	**Decorator @timer**	utils/decorators.py	Measures execution time with time.perf_counter() on every call
07	**Decorator factory @validate**	utils/decorators.py	Parameterised factory — returns a decorator that checks required keys
08	**Closure**	modules/insights_engine.py	make_threshold_checker(lo, hi) returns check() that closes over lo/hi
09	**Abstract Base Class**	modules/validation.py + 7	ABC + @abstractmethod enforces interface on all concrete subclasses
10	**Multiple Inheritance**	modules/validation.py	UserFormValidator(BaseValidator, SerializableMixin, LoggableMixin, ReprMixin)
11	**MRO (C3 linearisation)**	All multi-inherit classes	Python resolves method order — inspect with ClassName.__mro__
12	**Operator Overloading __add__**	modules/processing.py	DataProcessor + DataProcessor merges two DataFrames with pd.concat
13	**Operator Overloading __sub__**	modules/comparison_engine.py	snap_a - snap_b triggers ComparisonEngine.compare() automatically
14	**Mixin: SerializableMixin**	utils/mixins.py	Adds to_dict() · to_json() · from_dict() to any inheriting class
15	**Mixin: LoggableMixin**	utils/mixins.py	Adds log_event() · get_log() · clear_log() — per-instance logging
16	**Mixin: ReprMixin**	utils/mixins.py	Adds __repr__ · __str__ · __eq__ for readable object representation
17	**Regular Expressions**	modules/validation.py	re.compile() pre-compiles all 4 field patterns at class load time
18	**Multithreading**	modules/threading_tasks.py	threading.Thread + Lock — 4 threads process row chunks concurrently
19	**Multiprocessing**	modules/multiprocessing_tasks.py	Pool.map() distributes stats work across CPU cores with spawn context
20	**JSON Serialization**	modules/serialization.py	json.dump() saves records · json.load() reads · delete filters and writes
21	**os · sys · datetime · math**	Throughout all modules	File paths, version info, timestamps, pi constant, cpu_count
22	**NumPy**	modules/processing.py	mean · median · std · percentile · skewness · kurtosis · corrcoef
23	**Pandas**	modules/processing.py	read_csv · read_json · DataFrame · merge · concat · value_counts
24	**Matplotlib**	modules/processing.py	bar · line · hist · heatmap · all Agg backend, saved as PNG

■ 12 API & Route Reference

All 19 Flask

Route	Method	Page	Description
/	GET	index.html	Home page — user input form and dataset upload dropzone
/process	POST	dashboard.html	Validates form, loads dataset, runs all 7 analysis steps, renders results
/preprocess	GET	preprocess.html	Preprocessing home — upload form + feature overview cards
/preprocess	POST	preprocess.html	Scans uploaded file and returns issue report (missing, outliers, duplicates)
/preprocess/transform	POST	preprocess.html	Applies selected transformations, returns before/after preview + download link
/preprocess/download/	GET	download	Serves the cleaned CSV file as a direct file download
/profile	GET	profile.html	Data profiler home — upload form
/profile	POST	profile.html	Generates full per-column profile with skewness, kurtosis and mini charts
/filter	GET	filter.html	Filter & search home — upload form + filter builder
/filter	POST	filter.html	Applies filter conditions + search + sort, shows results table
/filter/download/	GET	download	Serves the filtered CSV file as a direct file download
/heatmap	GET	heatmap.html	Correlation heatmap home — upload form
/heatmap	POST	heatmap.html	Computes Pearson correlation matrix, generates heatmap PNG, lists all pairs
/merge	GET	merge.html	Dataset merger home — upload two files form
/merge	POST	merge.html	Scans common columns OR performs the join and returns merged preview
/merge/download/	GET	download	Serves the merged CSV file as a direct file download
/compare	GET	compare.html	Dataset comparison home — upload Dataset A and Dataset B
/compare	POST	compare.html	Runs snap_a - snap_b operator overloading and shows full diff results
/history	GET	history.html	Shows all past processing records from data/datasets.json
/history/delete/	POST	redirect	Deletes a record from datasets.json and redirects back to history
/api/stats	GET	JSON	Returns all processing records as raw JSON — useful for external tools
/api/validate	POST	JSON	Live field validation endpoint — called by script.js as you type
/api/health	POST	JSON	Upload a file and get back the health score JSON without the full dashboard